

TECHNICAL PORTFOLIO

Next.js Full-stack Engineering Portfolio

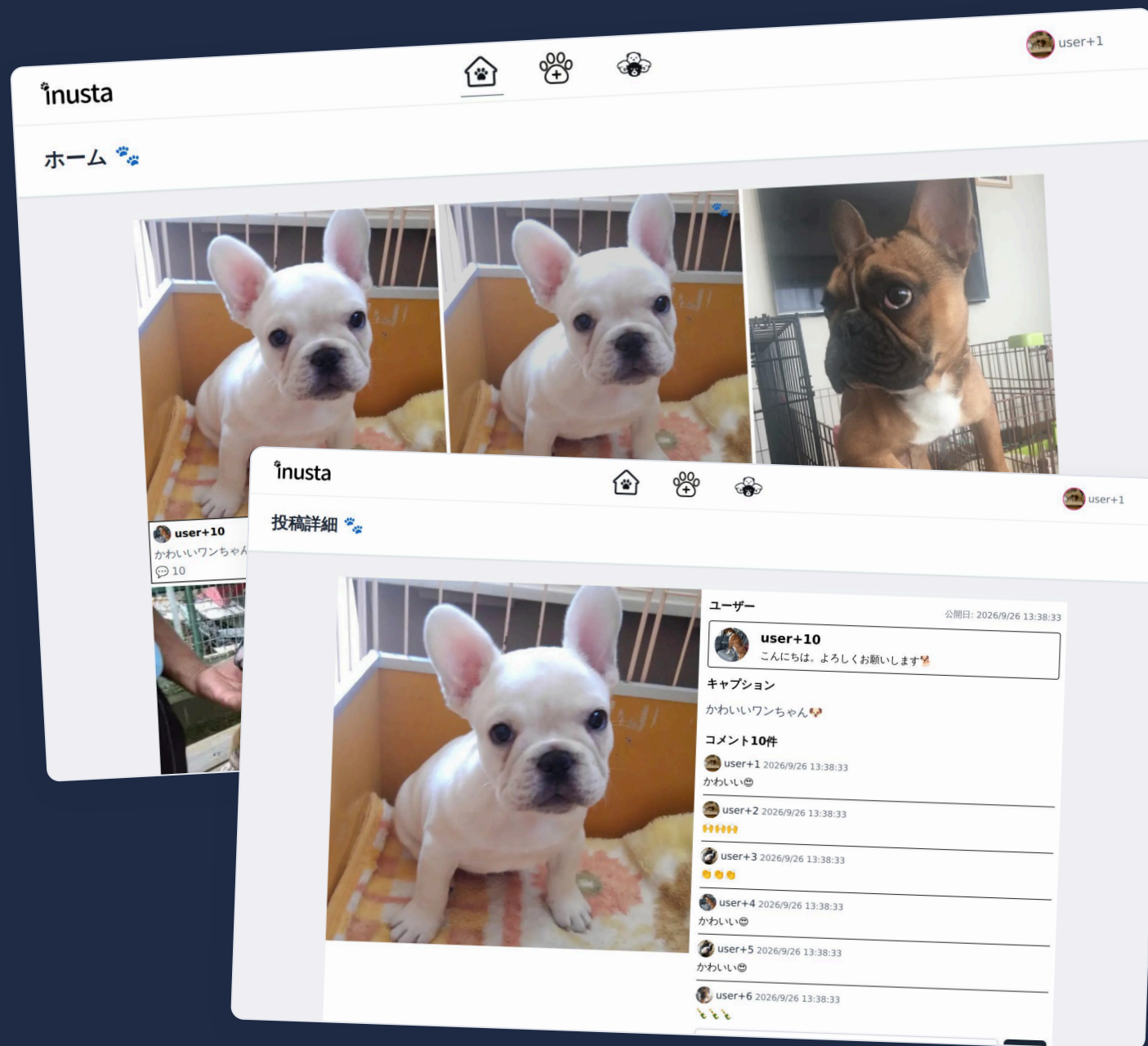
inusta — 犬の飼い主のための写真共有SNS

フロントエンドとバックエンドの経験を生かし、要件整理から認証・データ設計・デプロイまで一貫して実装します。

[氏名]

GitHub github.com/natsumi06/inusta-next

Demo inusta-next-delta.vercel.app



DBを持つWebサービスを、画面からデプロイまで一人で作り切る

inustaは、認証・投稿・コメント・画像保存を備えたフルスタックのSNSです。この1作品で示せる力を整理しました。

サービス概要

対象	愛犬の写真を気軽に残し、共有したい飼い主
目的	写真の投稿とコメントで、飼い主同士がゆるく交流できる場をつくる
形態	自社サービス型のWebアプリ（PC・スマホ対応）
公開	Vercelにデプロイし、PostgreSQLとVercel Blobに接続

14

画面（ルート）

9

Server Actions

3

DBモデル

この作品で示せること

認証・認可

Auth.jsのCredentials認証とMiddlewareでのルート保護。更新・削除はサーバー側で所有者を確認。

DB設計・CRUD

PrismaでUser・Post・Commentの1対多を定義。一覧・詳細・作成・編集・削除を実装。

画像アップロード

Vercel Blobに保存し、URLをDBへ。形式・サイズをサーバー側で検証。

UI / UX

Tailwind CSSでレスポンス対応。スケルトン表示、追加読み込み、削除前の確認モーダル。

運用・デプロイ

Vercelでの環境変数・DB接続・画像ドメイン設定を調整し本番公開。依存パッケージの脆弱性対応とNode.js v22への更新も実施。

技術スタック

Next.js 15

React 19

TypeScript

Tailwind CSS 4

Server Actions

Auth.js v5

bcrypt

Zod

Prisma 6

PostgreSQL

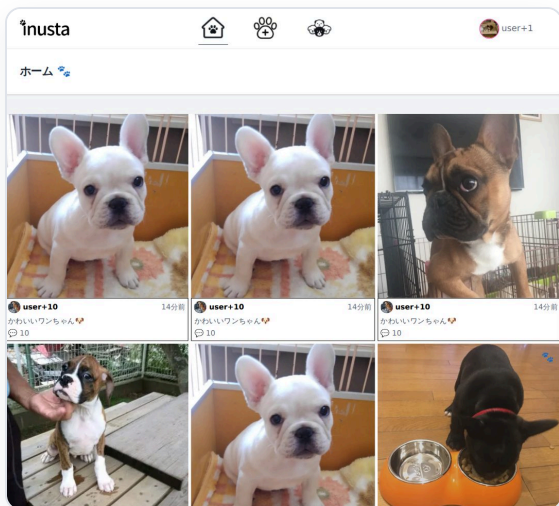
Vercel Blob

Vercel

主要画面と利用の流れ

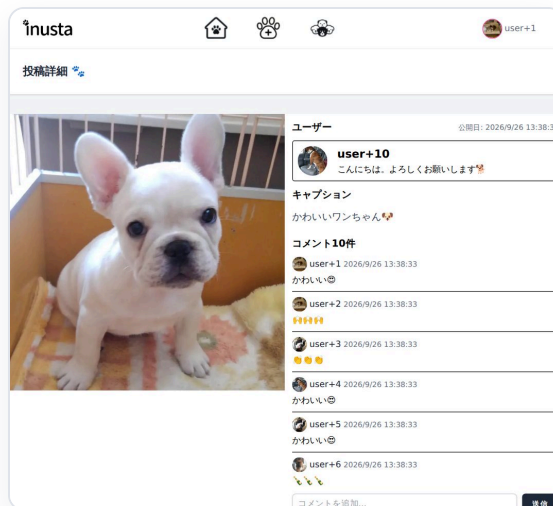
未ログイン時はトップ・登録・ログインのみ。ログイン後はナビゲーションから4つの主要画面を行き来します。

トップ → 会員登録 / ログイン → ホーム (新着投稿) ⇄ 投稿詳細・コメント ⇄ 投稿作成 ⇄ マイページ・プロフィール編集



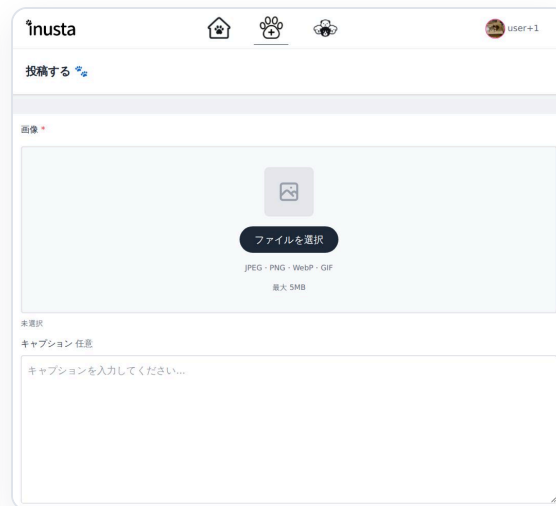
ホーム /posts

- 全ユーザーの新着投稿を一覧
- スクロールで20件ずつ追加読み込み
- コメント数・投稿者を表示



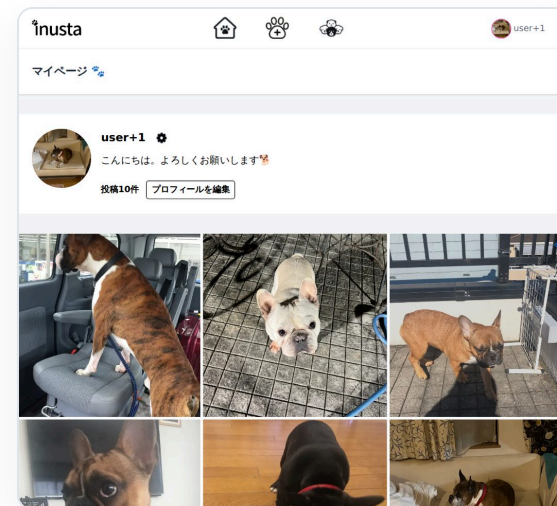
投稿詳細 /posts/[id]

- 画像・キャプション・投稿者
- コメント一覧と投稿フォーム
- 本人の投稿なら編集導線



投稿作成 /posts/create

- 画像選択とプレビュー
- JPEG・PNG・WebP・GIF／5MBまで
- キャプションは任意



マイページ /dashboard

- 自分の投稿をグリッド表示
- プロフィール編集・ログアウト
- 投稿ごとに編集・削除へ

その他の画面

投稿編集・削除 /posts/[id]/edit

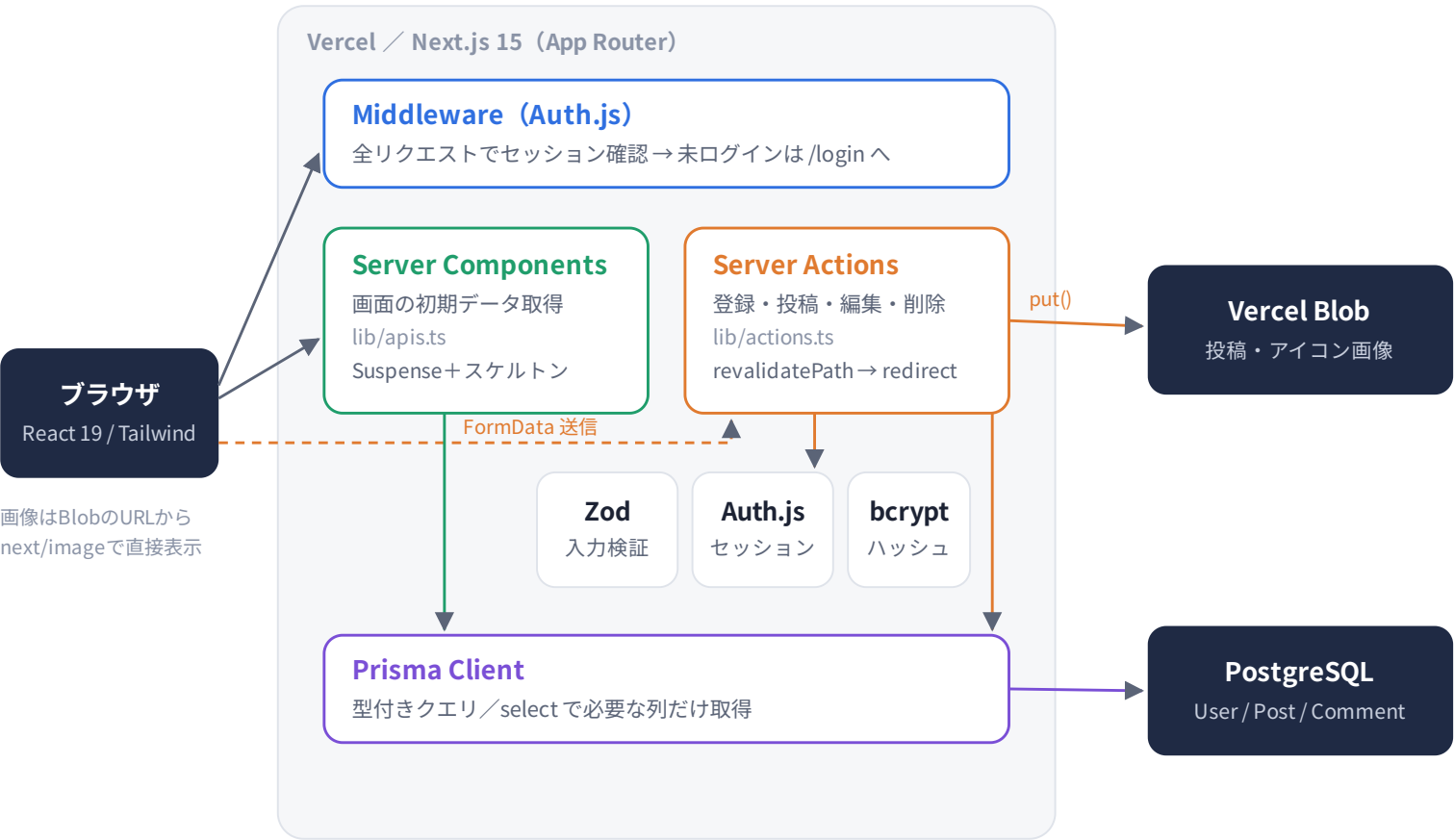
プロフィール編集 /profile

ユーザー一覧・詳細 /users

登録完了 /register/complete

技術構成：読み取りはServer Components、書き込みはServer Actions

APIを別サーバーに立てず、Next.js（App Router）の中でDB・ストレージ・認証までつなげています。

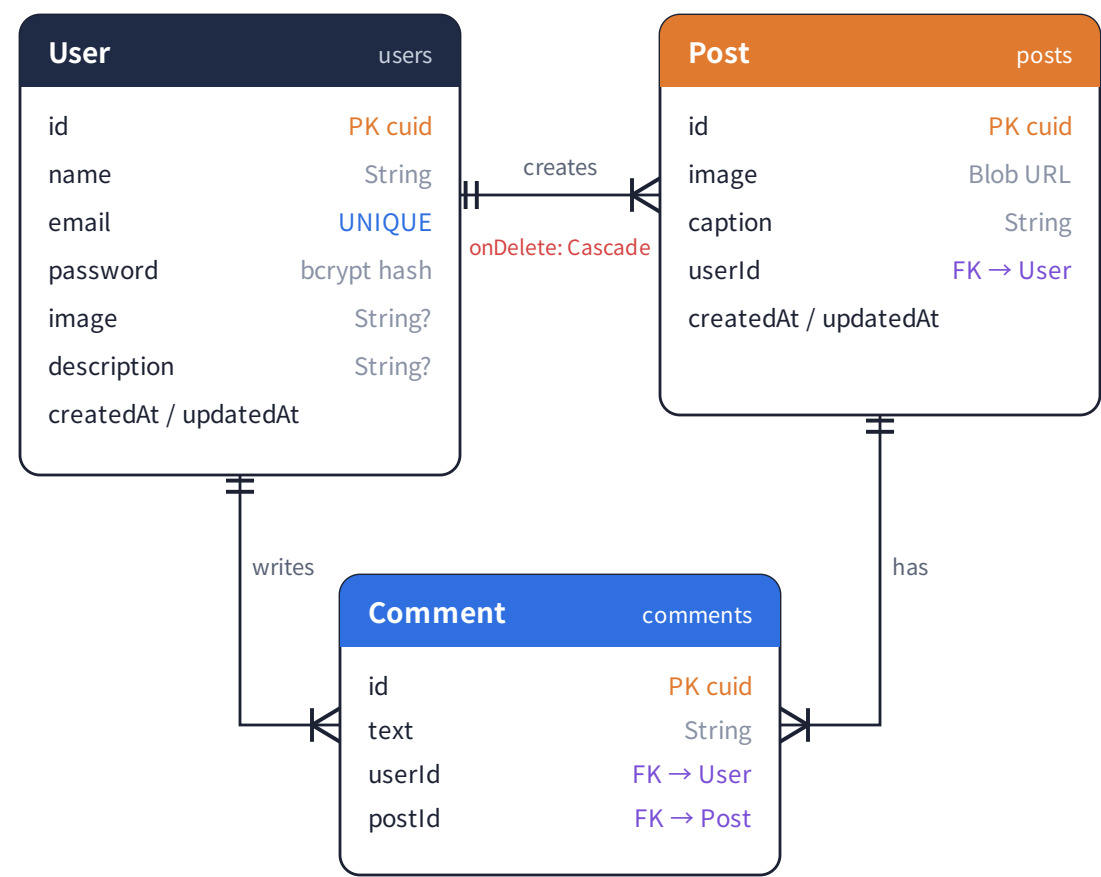


技術	役割
Next.js / React	画面とサーバー処理を1つのアプリで実装
Middleware	ログイン必須ページへの未認証アクセスを遮断
Server Actions	フォーム送信を受け、検証・保存・再描画まで行う
Zod	登録・ログイン入力の形式を実行時に検証
Auth.js / bcrypt	メール+パスワード認証、ハッシュ照合
Prisma	スキーマ定義から型付きのDB操作を生成
PostgreSQL	ユーザー・投稿・コメントを保存
Vercel Blob	画像ファイルを保存し公開URLを返す

※ 例外として、ヘッダーのユーザー表示用に Route Handler GET /api/me を1本用意しています。

データ設計と、機能ごとの処理

3つのモデルを1対多でつなぎ、ユーザーや投稿を削除すると関連データも連鎖削除される構造にしました。



機能	主な処理
会員登録	Zod検証 → メール重複確認 → bcryptでハッシュ化 → User作成 → 自動ログイン
ログイン	Zod検証 → Userを取得 → bcrypt.compare → セッション発行
投稿作成	認証 → 画像の形式・サイズ検証 → Blob保存 → Post作成
投稿編集・削除	投稿ID と ログインユーザーIDの両方で絞り込んで更新・削除
コメント	投稿の存在確認 → Comment作成 → 詳細ページを再検証
プロフィール	name・自己紹介を更新、画像があればBlobへ差し替え
新着一覧	skip / take で20件ずつ取得、_count でコメント数を同時取得

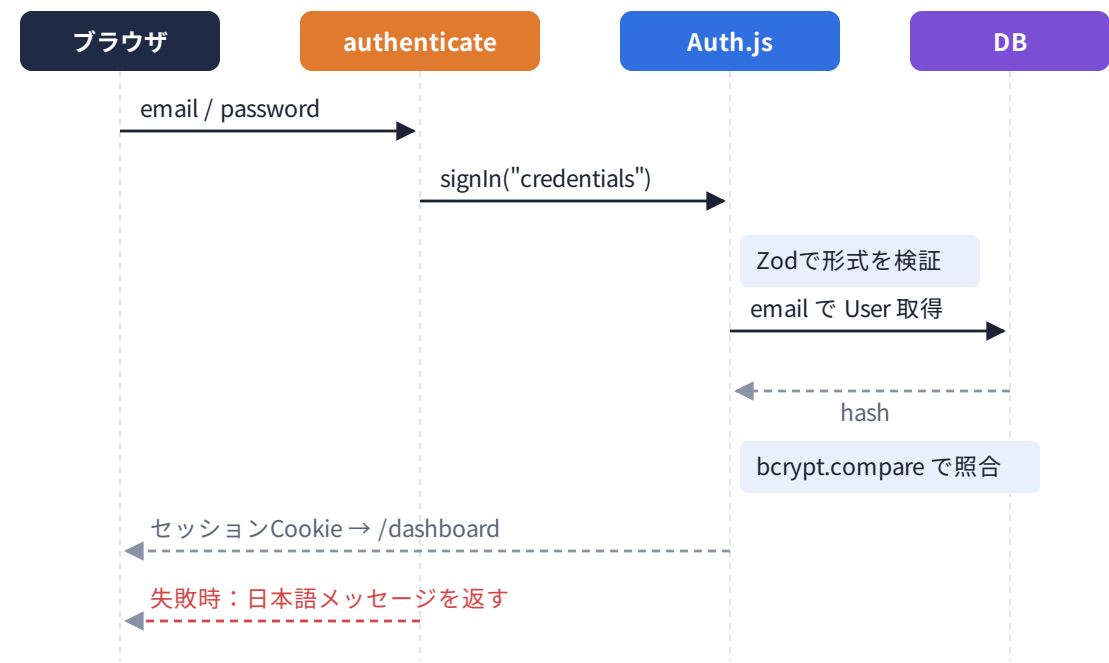
設計のポイント

- 画像本体はDBに入れず、BlobのURLだけを保存
- 一覧取得は select で表示に必要な列だけを返し、パスワード等を画面側に渡さない

認証・認可：「ログインしているか」と「本人の投稿か」を分けて守る

画面の入口はMiddlewareで、データの書き換えはServer Actionの中で、それぞれサーバー側で確認しています。

1 ログイン（認証）



2 ページ単位のアクセス制御（Middleware）

アクセス先	未ログイン	ログイン済み
/ ・ /login ・ /register	表示	/dashboard へ
上記以外（投稿・マイページ等）	/login へ	表示
/register/complete	登録直後の遷移を妨げないよう常に表示	

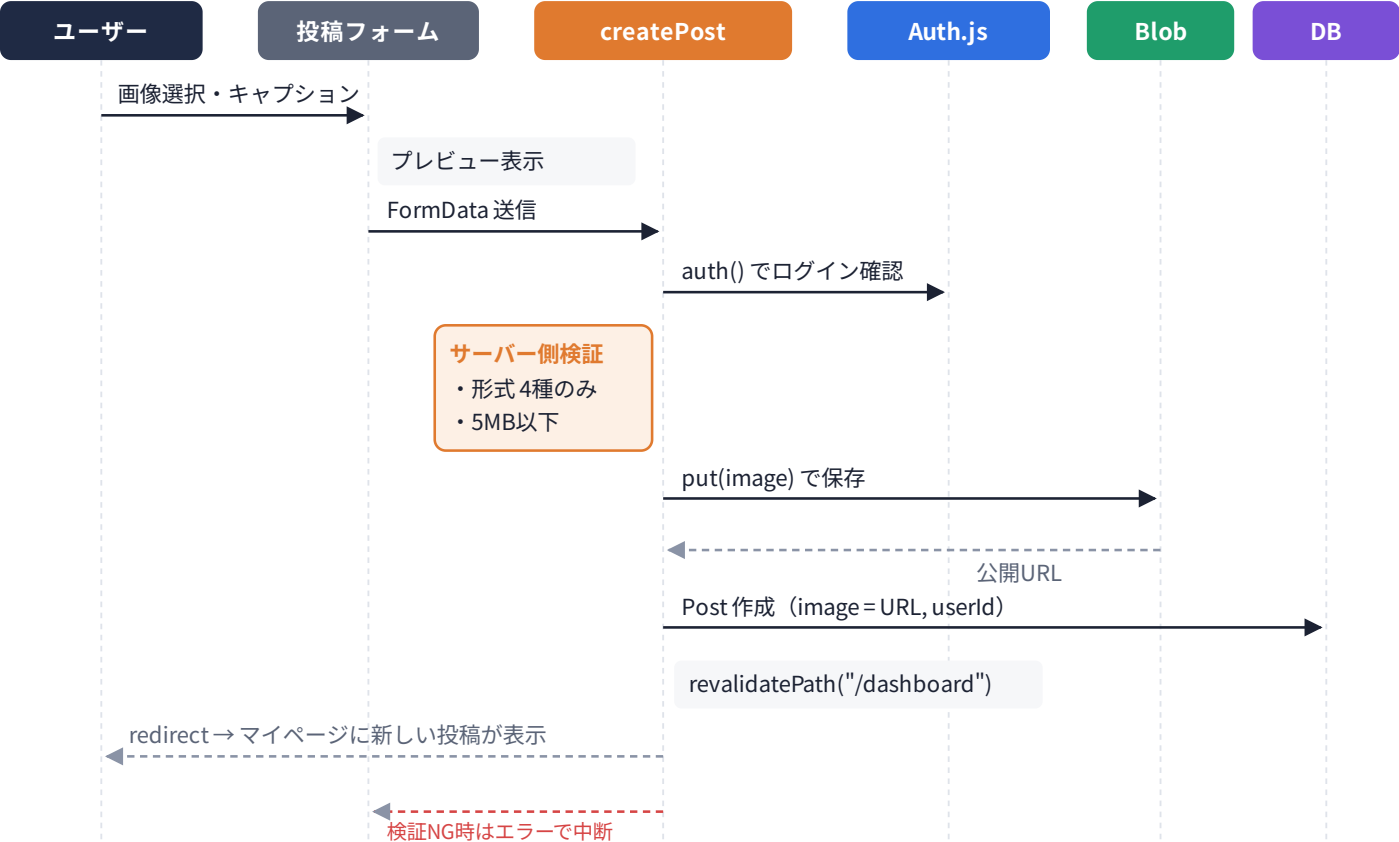
3 データ単位の認可（Server Action）

編集・削除は、投稿IDだけでなく**ログインユーザーのIDも条件に含めて**DBを操作します。画面のボタンを隠すだけでは、フォームを直接送られると防げないためです。

```
// lib/actions.ts (抜粋)
const session = await auth();
const user = await prisma.user.findFirstOrThrow({ where: { email } });
await prisma.post.delete({
  where: { id, userId: user.id }, // 他人の投稿は一致しない
});
```

代表処理：画像付き投稿の作成

フォーム送信から画面更新まで、1つのServer Actionの中で「認証 → 検証 → 保存 → 再描画」を順に行います。



バリデーションの置き場所

対象	ブラウザ	サーバー
投稿画像	形式・5MB・ プレビュー	形式・サイズ
会員登録	—	Zod：必須・形式・8文字以上・確認一致・重複
ログイン	—	Zod + bcrypt
コメント	—	未入力チェック

ブラウザ側は使いやすさのため、最終判断は必ずサーバー側で行う方針です。

Blobを先に保存する理由

Postの image は必須項目のため、先にBlobへ保存してURLを確定させてからレコードを作成します。これにより「画像のない投稿」がDBに残りません。

問題解決：見つけた課題と、その対応

実装・デプロイ中に発生した課題を「課題・対応・結果」の3行で整理しました。

セキュリティ

他人の投稿を書き換えられないようにする

課題 編集・削除ボタンを本人にだけ表示しても、投稿IDを指定したリクエストを直接送れば操作できてしまう。

対応 Server Actionでセッションからユーザーを特定し、`where: {id, userId}`で更新・削除するよう変更。

結果 所有者以外の操作はDBで一致せず失敗し、画面の作りに依存せずサーバー側で守れるようになった。

不具合修正

投稿時に選んだ画像が消える

課題 プレビュー表示に切り替える際にfile inputがDOMから外れ、送信時に画像が含まれないことがあった。形式・サイズの制限もなかった。

対応 inputを常に描画したまま見た目だけ切り替え、画像削除時は値をリセット。Server Actionに形式・5MBの検証を追加。

結果 画像が確実に送信され、想定外のファイルはサーバーで弾かれるようになった。

デプロイ

本番環境でDB・画像が繋がらない

課題 Vercelで発行される接続用の環境変数名とPrismaの参照先が食い違い、本番でDBに接続できなかった。Blob画像もnext/imageで表示できなかった。

対応 ログで実際の値を確認しながら接続先をDATABASE_URLに統一。BlobのドメインをremotePatternsに登録。

結果 ローカルと本番で同じ設定名で動くようになり、画像も最適化された状態で配信できた。

設計上の判断

「なぜその方法を選んだか」を説明できるように、主な判断を残しています。

構成 REST APIを作らずServer Actionsを採用

画面とサーバー処理を同じアプリ内に置けるため、APIの型定義やfetch処理を二重に書かずに済みます。読み取りはlib/apis.ts、書き込みはlib/actions.tsにまとめ、役割で分けました。

表示 Suspense＋スケルトンで待ち時間を見せる

DB取得を行うコンポーネントをSuspenseで囲み、読み込み中はレイアウトに合わせたスケルトンを表示。ヘッダーやナビゲーションは先に表示されます。

データ 必要な列だけを取得する

Prismaのselectで画面ごとに取得列を指定し、パスワードハッシュなど不要な値がクライアントへ渡らないようにしています。コメント数は_countで一覧と同じクエリ内で取得。

UX 無限スクロールは重複を防いで追加

初回20件をサーバーで描画し、末尾が見えたらIntersectionObserverでServer Actionを呼んで次の20件を取得。読み込み中フラグで二重リクエストを防ぎ、IDで重複を除外して結合します。

安全 取り消せない操作は一度確認する

投稿削除は「Danger Zone」として編集画面の下部に分離し、確認モーダルを挟んで実行。プロフィールや投稿の編集も送信前に内容を確認できるようにしました。

開発 シードデータで画面を確認しやすく

Prismaのseedで10ユーザー・100投稿・1,000コメントを投入できるようにし、一覧・追加読み込み・詳細の表示確認を実データに近い量で行えるようにしました。

品質への取り組み・AI活用・今後の改善

現状できていることと、まだ足りないことを分けて整理しています。

品質

- Zodによる登録・ログイン入力の検証
- bcryptによるパスワードのハッシュ保存
- サーバー側での所有者確認（編集・削除）
- 画像の形式・サイズ制限
- TypeScript・ESLint・本番ビルドの通過を確認
- 依存パッケージの脆弱性対応、Node.js v22対応
- PC・スマホ幅での表示確認

AI活用

Claude Codeを「調査と実装案の作成」に使い、採用の判断は自分で行っています。

- 1 不具合の原因調査・修正案の作成を依頼
- 2 差分と影響範囲を自分で読んで確認
- 3 Lint・ビルド・画面操作を確認してから採用

例：投稿画像が消える不具合の原因特定、ESLint循環参照エラーの解消、UI改善項目の洗い出し。

今後の改善

- **自動テスト**：Server Actionsの単体テスト、Playwrightで主要フローのE2E
- **CI**：GitHub ActionsでLint・型チェック・ビルドを自動実行
- **検証の統一**：キャプション・コメント・プロフィール画像にもZodと同じ制限を適用
- **監視**：エラー監視・ログ収集の導入
- **運用**：画像差し替え時の旧Blob削除、Docker化

ソースコード・デモはこちら

github.com/natsumi06/inusta-next / inusta-next-delta.vercel.app